Sistemas Inteligentes

Práctica 1 - Satisfacción de Restricciones

Universidad de Alicante

Nael Alba Laira
48804733K
nal22@alu.ua.es

Índice

1. Explicación de la clase Variable ..... Pág. 3

2. ¿Cómo se han tratado los carácteres fijos? ..... Pág. 4

3. Especificación formal del problema (Problema Pequeño) ..... Pág. 5

4. Traza del algoritmo Forward Checking ..... Pág. 6

5. Traza del algoritmo AC3 ..... Pág. 7

6. Experimentación y Análisis de Tiempos ..... Pág. 8

7. Dificultades encontradas ..... Pág. 14

8. Conclusiones obtenidas ..... Pág. 15

9. Uso de IAs Generativas ..... Pág. 15

10. Bibliografía ..... Pág. 15

1. Explicación de la clase Variable

La clase variable representa los elementos a resolver en nuestro CSP. Cada variable corresponde a una palabra completa del crucigrama ya sea en horizontal o vertical.

Atributos de la clase

Dada la libre elección de la estructura de la clase variable dependiendo de la implementación he optado por la siguiente lista de atributos:

- **Nombre:** Identificador único de la variable. Se construye en base a su orientación y coordenadas.
- **Tipo:** Orientación de la variable, siendo h para horizontal, v para vertical y u para las variables de longitud 1 (ya que son tratadas de otra forma en mi código).
- **Pos:** Coordenada matricial de inicio (fila, columna).
- **Long:** Longitud de la variable, dada por el número de celdas.
- **Robligatorias:** Lista de tuplas (pos, caracter) correspondientes a las letras fijas asignadas a la variable.
- **Dominio:** Conjunto de palabras candidatas del almacén que cumplen con la longitud y las restricciones.
- **Ractuales:** Lista de tres-tuplas (vecino, pos_mia, pos_vecino) que se encarga de gestionar las restricciones dinámicas que se van generando a medida que avanza la ejecución.
- **Palabra_Actual:** Valor asignado a la variable durante la ejecución.

Métodos Implementados

- **calcularDominio(almacen):** Filtra las palabras de longitud exacta que coinciden con los carácteres fijos obligatorios. Se ejecuta al crear las variables.
- **actualizarDominio():** Filtra el dominio de la variable en tiempo de ejecución, comprobando que las palabras cumplan con las nuevas restricciones generadas durante la ejecución del programa. Sólo comprueba las restricciones con las variables ya implementadas.

2. ¿Cómo se han tratado los carácteres fijos?

Las casillas de carácteres fijos son las que representan las restricciones iniciales, puestas por el usuario antes de comenzar la resolución del programa.

En mi implementación se han tratado procesando las variables durante su creación. Para ello creamos una lista de restricciones en *construir_variables*

Durante el escaneo inicial de una variable en el cual se determina su orientación, longitud… Creamos una lista de restricciones a la cual se le añaden los carácteres y la posición de las celdas que al no estar llenas, tampoco están vacías, ya que contienen una letra en su interior.

En el constructor de la clase variable se llama al método de ***calcularDominio()*** que descarta todas las palabras del dominio que no cumplan con la restricción de tener el carácter de la restricción en la posición adecuada. Para calcular la posición adecuada utilizamos la posición de la restricción menos la inicial, lo que nos da el index de desplazamiento.

De este modo, las restricciones no requieren comprobación activa de su cumplimiento, siendo innecesarias sus comprobaciones posteriores.

3. Especificación formal del problema (Problema Pequeño)

3.1 Minidiccionario de palabras (A)

Diccionario reducido A ={‘SOL’, ‘SUR’, ‘SAL’, ’ORO’, ‘OSO’, ‘RIO’}

3.2 Tupla formal <V, E, c, l, a>

La formalización del problema se define mediante:

- **V = {H0, H1, V0, V1}**
  - H0, Variable horizontal en la fila 0, columnas 0 a 2
  - H1, Variable horizontal en la fila 2, columnas 0 a 2
  - V0, Variable vertical en la columna 0, filas 0 a 2
  - V1, Variable vertical en la columna 2, filas 0 a 2

  Celdas intermedias vacías, y la celda (1, 1) llena
- **l = {l(H0) = 3, l(H1) = 3, l(V0) = 3, l(V1) = 3}** (Longitudes)
- **c (Conjunto de celdas):**
  - c(H0)={(0,0),(0,1),(0,2)}
  - c(H1)={(2,0),(2,1),(2,2)}
  - c(V0)={(0,0),(1,0),(2,0)}
  - c(V1)={(0,2),(1,2),(2,2)}
- **Dominios iniciales:**

  D(H0) = D(H1) = D(V0) = D(V1) = {‘SOL’, ‘SUR’, ‘SAL’, ’ORO’, ‘OSO’, ‘RIO’}
- **Restricciones de cruce (E)**
  - Cruce (H0, V0) en (0, 0)
  - Cruce (H0, V1) en (0, 2)
  - Cruce (H1, V0) en (2, 0)
  - Cruce (H1, V1) en (2, 2)
- **Restricción global:** No repetir palabra

4. Traza del algoritmo Forward Checking

Tras cada asignación se **podan los dominios de las variables aún sin asignar** (por cruce y por no repetir palabra). Si algún dominio queda vacío (*wipe-out*), se deshace y se prueba el siguiente valor.

1. H0 = SOL. Como H0[2] = L, V1 debería empezar por L y ninguna palabra lo hace: **D(V1) =** ∅. *Fallo, backtrack.*

2. H0 = SUR. V0 debe empezar por S → {SOL, SAL}. V1 debe empezar por R → {RIO}. H1 queda {SOL, SAL, ORO, OSO, RIO}. *Sin fallo, se continúa.*

2.1. V0 = SOL. Como V0[2] = L, H1 debería empezar por L: **D(H1) =** ∅. *Fallo, backtrack.*

2.2. V0 = SAL. Igual que antes (V0[2] = L): **D(H1) =** ∅. *Fallo. D(V0) agotado, se vuelve a H0.*

3. H0 = SAL. H0[2] = L → **D(V1) =** ∅. *Fallo, backtrack.*

4. H0 = ORO. V0 debe empezar por O → {OSO} (ORO ya usada). V1 también → {OSO}. *Sin fallo, se continúa.*

4.1. V0 = OSO. Se elimina OSO de V1 por no repetir: **D(V1) =** ∅. *Fallo. D(V0) agotado, se vuelve a H0.*

5. H0 = OSO. V0 → {ORO} y V1 → {ORO}. *Sin fallo, se continúa.*

5.1. V0 = ORO. Se elimina ORO de V1: **D(V1) =** ∅. *Fallo. D(V0) agotado, se vuelve a H0.*

6. H0 = RIO. V0 debe empezar por R y la única palabra con R es RIO, ya usada: **D(V0) =** ∅. *Fallo. D(H0) agotado.*

El problema no tiene solución. FC lo detecta tras **10 asignaciones**.

5. Traza del algoritmo AC3

AC3 no asigna valores, elimina los que no tienen apoyo.

Cola inicial: (H0,V0), (V0,H0), (H0,V1), (V1,H0), (H1,V0), (V0,H1), (H1,V1), (V1,H1).

Si un dominio cambia, se reencolan los arcos que apuntan a esa variable.

1. (H0,V0) y **2. (V0,H0).** Las primeras letras posibles son {S, O, R} en ambas variables, así que todos los valores tienen apoyo. *Sin cambios.*

3. (H0,V1). H0[2] = V1[0].

Las últimas letras de H0 son:

SOL→L ✗, SUR→R ✓, SAL→L ✗, ORO→O ✓, OSO→O ✓, RIO→O ✓.

Resultado: **D(H0) = {SUR, ORO, OSO, RIO}**. Se añade (V0,H0).

4. (V1,H0). V1 debe empezar por una última letra de H0, es decir {R, O}.

Se descartan SOL, SUR y SAL.

Resultado: **D(V1) = {ORO, OSO, RIO}**. Se añade (H1,V1).

5. (H1,V0). H1[0] = V0[2].

Las últimas letras de V0 son {L, R, O}, así que las palabras que empiezan por S no tienen apoyo.

Resultado: **D(H1) = {ORO, OSO, RIO}**.

6. (V0,H1). V0 debe terminar en una primera letra de H1, es decir {O, R}.

Se descartan SOL y SAL (terminan en L).

Resultado: **D(V0) = {SUR, ORO, OSO, RIO}**. Se añade (H0,V0).

7. (H1,V1) y **8. (V1,H1).** V1 solo termina en O y todas las palabras de H1 también.

Sin cambios.

9. (V0,H0) y **10. (H0,V0)** (reencolados). Todos los valores conservan apoyo.

Sin cambios. Cola vacía.

Dominios finales:

H0 = V0 = {SUR, ORO, OSO, RIO};

H1 = V1 = {ORO, OSO, RIO}.

AC3 reduce los dominios, pero **ningún dominio queda vacío**, así que **no detecta** la inconsistencia.

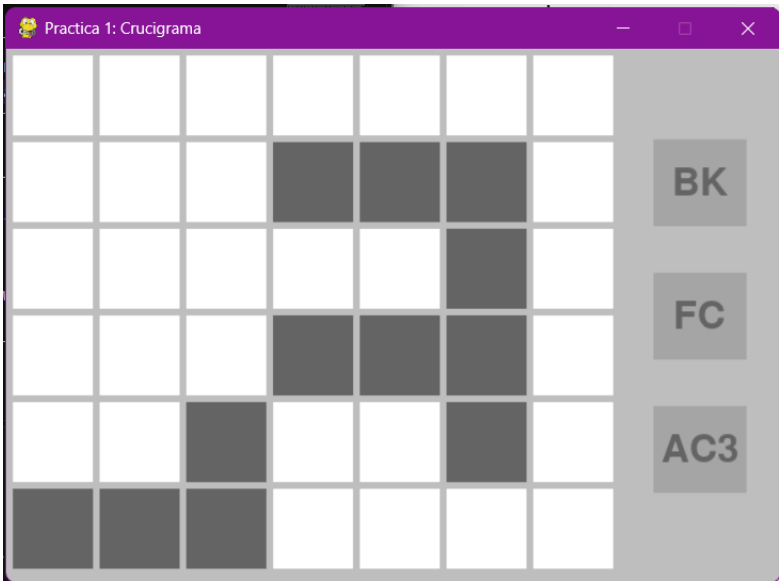
La restricción *no repetir palabra* no actúa mientras los dominios tengan más de un valor.

6. Experimentación y Análisis de Tiempos

Para el análisis de resultados y tiempos utilizamos dos diccionarios diferentes. d0 y d1 siendo d1 un diccionario con alrededor de 100 palabras y d0 el diccionario por defecto de la práctica.
Todos los tiempos de BK+AC3, BJ+AC3 y FC+AC3 es aislando los valores de AC3, es decir únicamente los valores de la ejecución del algoritmo tras le AC3.

Crucigrama 6x7 - Con Solución

python main.py --filas 6 --columnas 7 --dic d0.txt

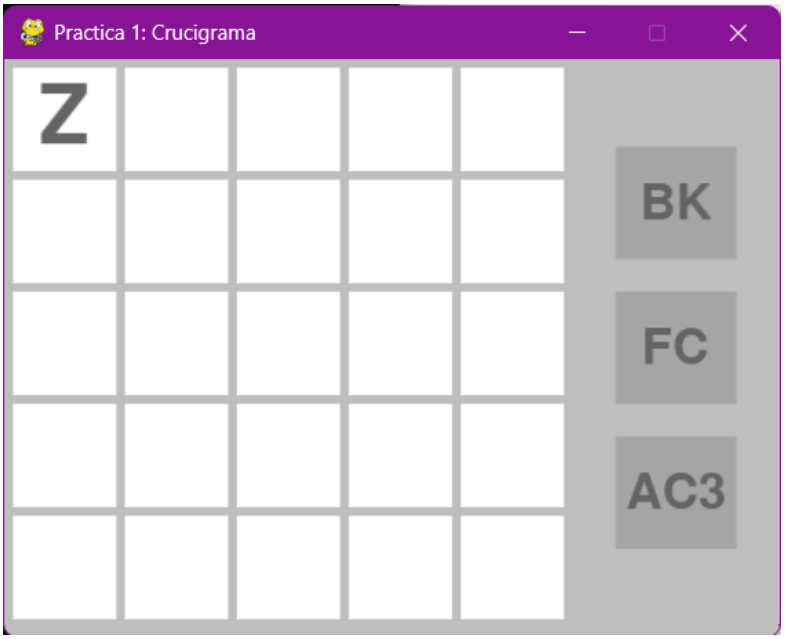


Algoritmo / Estrategia	Tiempo (d0.txt)	Tiempo (d1.txt)
BK	431.763 ms (0.43176 s)	No responde
BJ	1.035 ms (0.00104 s)	9.510 ms (0.00951 s)
FC	0.170 ms (0.00017 s)	0.322 ms (0.00032 s)
AC3	1.636 ms (0.00164 s)	2.004 ms (0.00200 s)
BK + AC3	0.104 ms (0.00010 s)	0.154 ms (0.00015 s)
FC + AC3	0.137 ms (0.00014 s)	0.153 ms (0.00015 s)
BJ + AC3	0.095 ms (0.00010 s)	0.120 ms (0.00012 s)

En este crucigrama podemos observar como el tiempo aumenta a medida que aumenta el diccionario, llegando en ciertos casos de backtracking a no responder el programa.

Crucigrama 5x5 - Sin Solución (Detectada por AC3)

python main.py --filas 5 --columnas 5 --dic d0.txt

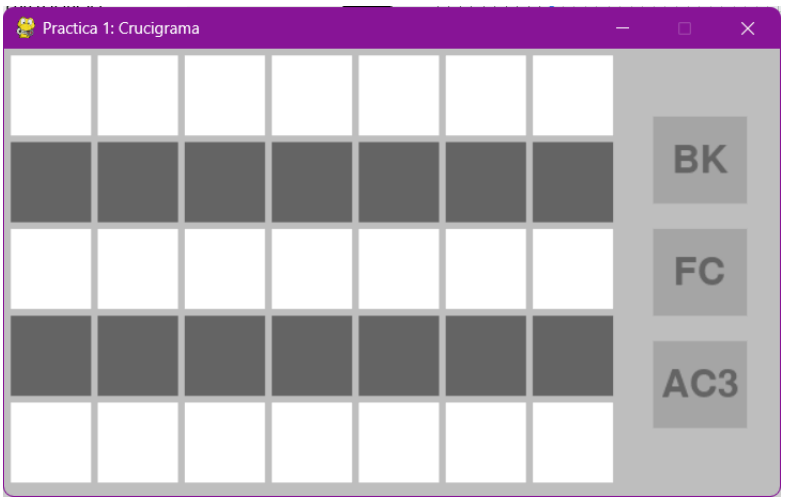


Algoritmo / Estrategia	Tiempo (d0.txt)	Tiempo (d1.txt)
BK	0.012 ms (0.00001 s)	0.007 ms (0.00001 s)
BJ	0.019 ms (0.00002 s)	0.014 ms (0.00001 s)
FC	0.006 ms (0.00001 s)	0.006 ms (0.00001 s)
AC3	2.320 ms (0.00232 s)	1.080 ms (0.00108 s)

No se ejecutan el resto de pruebas ya que al dar error AC3, es innecesario.
Este ejemplo nos muestra que en caso de tratar un crucigrama el cuál no tiene solución, la diferencia de tiempo va a encontrarse en el origen del problema, dependiendo de cuándo se detecte. El tamaño del diccionario varía el resultado en problemas cuyo error se encuentra más tarde.

Crucigrama 5x7 - Sin Solución para d0.txt (AC3 deja valores)

python main.py --filas 5 --columnas 7 --dic d0.txt

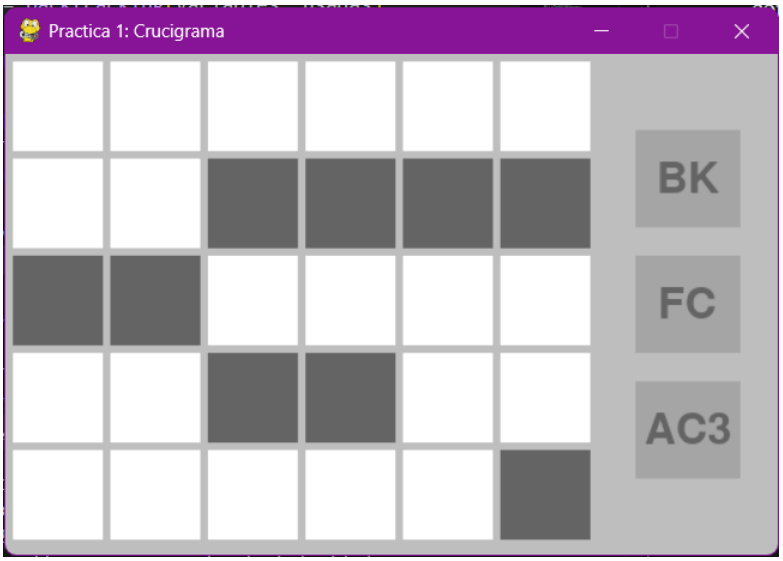


Algoritmo / Estrategia	Tiempo (d0.txt) - Sin Solución	Tiempo (d1.txt) - Solución
BK	0.035 ms (0.00004 s)	0.019 ms (0.00002 s)
BJ	0.049 ms (0.00005 s)	0.028 ms (0.00003 s)
FC	0.027 ms (0.00003 s)	0.013 ms (0.00001 s)
AC3	0.746 ms (0.00075 s)	0.666 ms (0.00067 s)
BK + AC3	0.057 ms (0.00006 s)	0.033 ms (0.00003 s)
FC + AC3	0.034 ms (0.00003 s)	0.019 ms (0.00002 s)
BJ + AC3	0.071 ms (0.00007 s)	0.034 ms (0.00003 s)

Este ejemplo nos muestra como un diccionario puede cambiar completamente la solución del problema, incluso mejorando los tiempos de error.
Para nuestro problema, el algoritmo AC3 cumple con su función pero no llega a detectar que el problema no cuenta con solución al quedarse todos los dominios con valores.
Se puede observar como, incluso teniendo que buscar una solución, en este caso el tiempo de los algoritmos aplicando el diccionario de mayor tamaño es menor. Esto se debe a que la solución se encuentra de manera directa unicamente asignando valores, sin llegar a cambiar ninguno por restricción.

Crucigrama 5x6 - Sin Solución (Falla tarde)

python main.py --filas 5 --columnas 6 --dic d0.txt

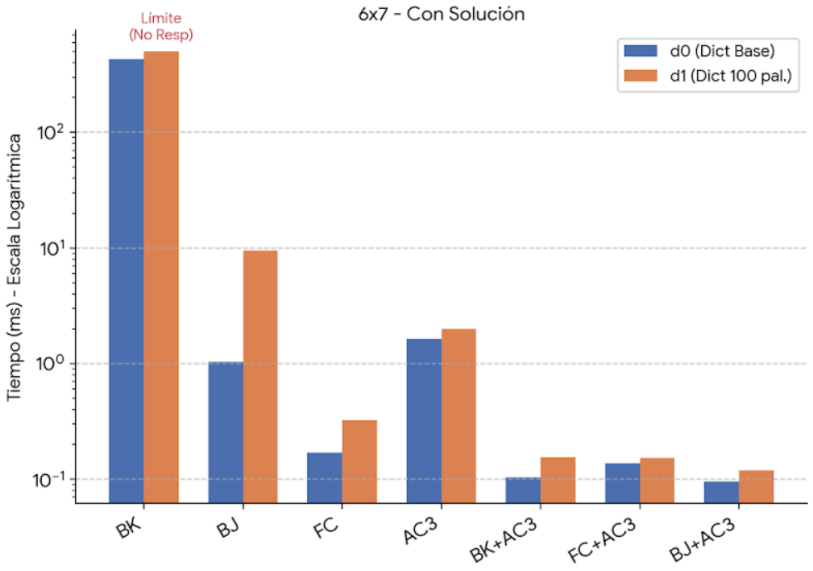


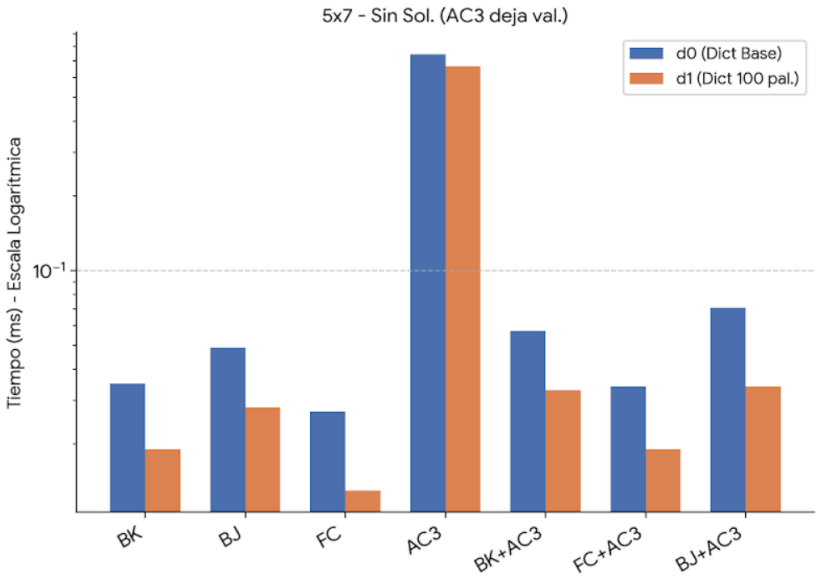
Algoritmo / Estrategia	Tiempo (d0.txt)	Tiempo (d1.txt)
BK	373.812 ms (0.37381 s)	No responde
BJ	18.378 ms (0.01838 s)	342.745 ms (0.34275 s)
FC	0.519 ms (0.00052 s)	18.111 ms (0.01811 s)
AC3	1.111 ms (0.00111 s)	1.476 ms (0.00148 s)
BK + AC3	0.063 ms (0.00006 s)	1.251 ms (0.00125 s)
FC + AC3	0.108 ms (0.00011 s)	0.668 ms (0.00067 s)
BJ + AC3	0.047 ms (0.00005 s)	0.257 ms (0.00026 s)

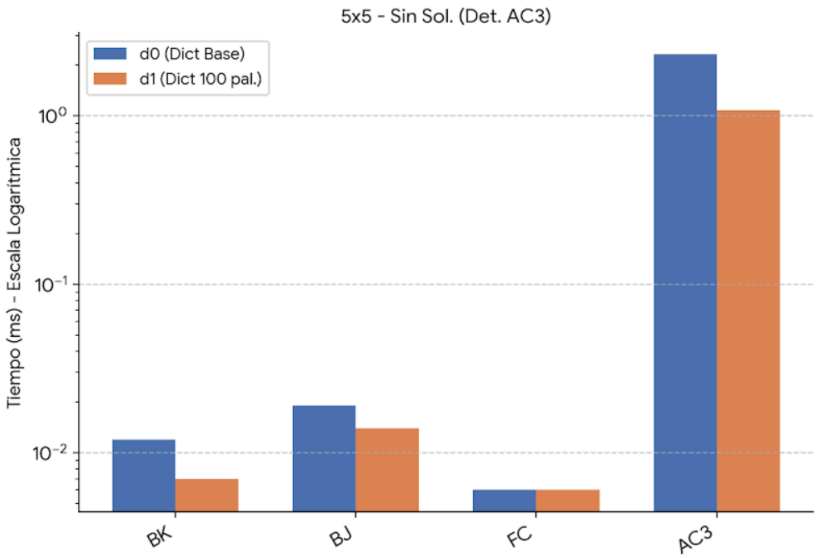
En este crucigrama comprobamos como pese a no haber solución, el tiempo es considerablemente mayor al de los tratados anteriormente sin solución, esto se debe a la cantidad de iteraciones que tienen que realizar los algoritmos para llegar a la conclusión de que no hay solución.
Se puede observar como el tamaño del diccionario hace que aumente el tiempo.

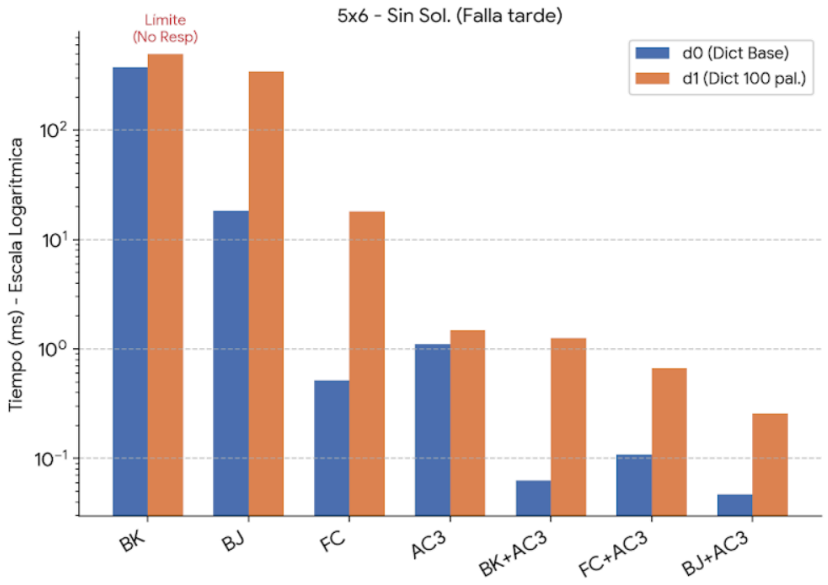
Gráficas

Las gráficas son representadas en escala logarítmica, ya que en escala lineal era imposible captar la diferencia de tiempos al ser tan alta entre BK y el resto.









Conclusiones del análisis

En general, se puede observar como el peor algoritmo es el Backtracking, tardando una cantidad considerable en problemas de mayor tamaño incluso llegando a no responder en algunos.

Por otra parte, pese a mejorarlo generando el algoritmo de Backjumping, esto no basta para conseguir ser el algoritmo más eficaz. Ya que incluso mejorando el tiempo y reduciéndolo una cantidad considerable no alcanza la del Forward Checking, el cual es el algoritmo que mejores resultados en tiempo consigue. Cambiando la cosa al aplicar AC3 donde hay casos en los que el más eficiente es el BJ.

AC3 reduce los tiempos de todos los algoritmos y puede llegar a encontrar la no solución en caso de que se quede sin dominios.

Podemos observar que hay diferentes condiciones que hacen que se aumente o reduzca el tiempo de resolución de los algoritmos.
Entre ellos se destaca la cantidad de variables, ya que si vemos el crucigrama ***5x7*** que cuenta unicamente con 3 variables y lo comparamos con el ***5x6*** que cuenta con 12 variables se puede observar una gran diferencia en tiempos.
Otra condición es la cantidad de cruces, ya que esto hace que los algoritmos tengan que retroceder y cambiar valores continuando las iteraciones, se puede observar en los mismos ejemplos que el anterior.
El cambio de diccionario, aumentando la cantidad de palabras posibles no solo puede hacer que haya solución en un crucigrama que con un dominio menor no habría *(Como en el ejemplo del crucigrama **5x7**)* también puede ser que los tiempos se reduzcan, no siempre aumentan con el tamaño del diccionario, ya que puede darse la casualidad de que la ordenación de este diccionario sea una solución para dicho crucigrama.

7. Dificultades encontradas

La principal dificultad encontrada durante la realización de la práctica ha sido el lenguaje, ya que anteriormente no había programado nada en python y no manejaba correctamente tuplas, funciones…

Al principio hice un esquema visual de la clase variable e iba añadiendo atributos según creía más conveniente, llegando a crear estructuras raras como Ractuales, Robligatorias o la tres-tupla que se crea al construir las variables ya que las trato de forma diferente según su orientación.

La dificultad principal ha venido en la reconstrucción de las tuplas en el algoritmo AC3, debido a que al querer poner la tupla que verifique la restricción en el sentido opuesto tenía que jugar con las diferentes posiciones de los atributos ya creados en la clase.

Otra dificultad que encontré fue durante la creación del algoritmo de Forward Checking ya que tras la asignatura de ADA aprendí a realizar Backtracking utilizando llamadas recursivas a base de booleanos, y al no tener la función de es_consistente que muchos compañeros han diseñado, me las tuve que ingeniar con la reconstrucción de los dominios modificados. Optando por hacerlos de forma local, ya que cada variable guardaba su dominio anterior y en caso de fallar se reconstruía.

La última fue realizando el Backjumping, ya que al no encontrar una manera de implementarlo correctamente, acabé optando por utilizar una estructura para controlar el orden de las palabras, un diccionario con parejas de palabra-restricción para las usadas y un conjunto de conflictos de cada variable.

8. Conclusiones obtenidas

El algoritmo de Backtracking por si solo es una solución inviable a los problemas de mayor tamaño.

La propagación (Forward Checking) es, de los que hemos tratado, el algoritmo con mejores resultados y el algoritmo AC3 es una muy buena combinación para cualquiera de los algoritmos tratados.

9. Uso de IAs Generativas

Durante la realización de la práctica el uso de IA se ha limitado a comprobaciones y solución de fallos concretos. Además de diferentes guías sobre el lenguaje o funciones de este.

Durante la realización del Backjumping, le pedí una guía sin implementación sobre como realizar el algoritmo y acabé usando sus atributos recomendados, los mencionados anteriormente del diccionario de parejas y el orden.
También me ayudó añadiendo ***deque*** en el algoritmo de AC3 para evitar aumentar la complejidad con otros tipos de búsqueda.
Por último, durante la realización de la memoria le pedí el diccionario d1 y una serie de pruebas que cumplieran con los requisitos que le pedía, siendo estos los casos mostrados anteriormente en el punto 6.

10. Bibliografía

- Apuntes Sistemas Inteligentes Teoría - Tema 3.2 CSP

- Apuntes Sistemas Inteligentes Prácticas - Introducción Python

- Documentación Python
  https://docs.python.org/3/library/collections.html#collections.deque
- Gemini 3.1 Pro